CHAPTER 1 • GETTING STARTED WITH SNOWFLAKE ~15 Min

Student

Sir, I want to become a **data engineer**. I already know **SQL** (from the Week 1-2 SQL deep dive) and **data modeling** (from Weeks 2-3 with the Mango Mart star schema). Everyone keeps telling me the next skill I need is **Snowflake**. What is it, really, and where do I begin?

Claude • AI Assistant

Perfect launch point. You are exactly where thousands of engineering students find themselves - you know how to write good SQL, you understand what a fact table and a dimension are, but the word 'Snowflake' is still a fog.

This entire chapter is designed to lift that fog before we do any deep technical work. By the end of Chapter 1 you will:

1. Know what a **cloud data warehouse** is and why the industry moved away from on-prem warehouses.
2. Understand Snowflake's foundational idea - the **separation of storage and compute** - and why it changes what is economically possible in analytics.
3. Have a real Snowflake account of your own, running on the **30-day free trial**, with **Snowsight** (web UI) and a starter database ready.
4. Have **SnowSQL** installed and connected - the command-line client that turns Snowflake into a scriptable, automatable platform.
5. Own the **object hierarchy** - account, database, schema, table, warehouse, role - so nothing feels arbitrary when we start writing production code from Chapter 2 onward.

CONCEPT CARD

Snowflake is a cloud-native, fully managed, columnar, MPP data platform delivered as software-as-a-service on AWS, Azure, and Google Cloud. It is the leading **cloud data warehouse** in the industry and has expanded into semi-structured data, data sharing, machine learning (**Snowpark**), and generative AI (**Cortex**).

Every subject in the rest of this refcard rests on the five foundations we build in this chapter.

ANALOGY

If SQL is your **language** and data modeling is your **architectural blueprint**, then Snowflake is the **construction site with a full toolkit** - the ground you build on, the

cranes you use, the safety systems, and the utilities. Chapters 2 through 5 are you learning to operate the equipment.

Chapter 1 is you walking around the site and learning the lay of the land.

REAL WORLD APPLICATION

In the past two years, Snowflake has become the default answer to 'where do we put our analytics data?' at companies ranging from Indian D2C startups to global Fortune 500s. Roles tagged 'Snowflake developer', 'Snowflake data engineer', and 'Snowflake architect' consistently pay 20-30% above generic SQL roles on Indian job boards.

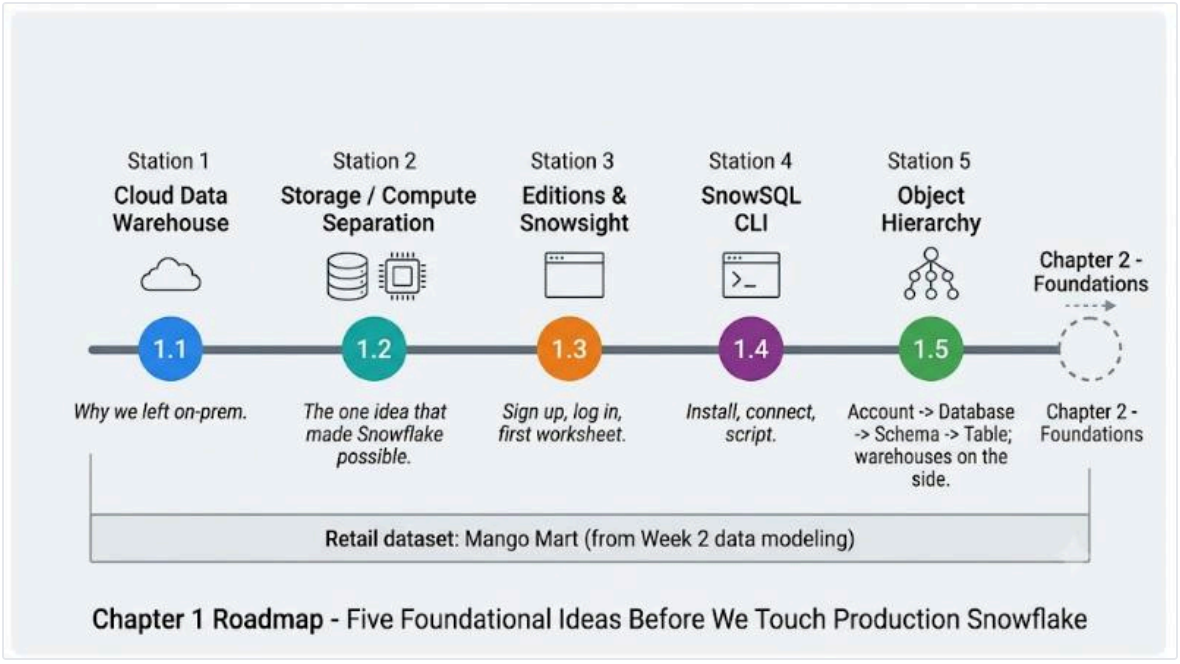
The **SnowPro Core certification** - which this refcard prepares you for - is one of the most in-demand data credentials of the current market. What you learn here is directly economically valuable.

Student

Great - let's start from zero. What is a cloud data warehouse?

Claude • AI Assistant

Follow me into subsection 1.1.



Chapter 1 Roadmap - Five Foundational Ideas Before We Touch Production Snowflake

Chapter 1 at a Glance - Subsection Learning Outcomes

Subsection	Title	Core Question Answered	You Can Now…
1.1	What is a Cloud Data Warehouse?	Why did the industry move to the cloud?	Explain OLTP vs OLAP; place Snowflake in the modern data stack.
1.2	Why Snowflake - Separation of Storage and Compute	What one idea makes Snowflake unique?	Explain multi-cluster shared-data; distinguish scaling up from scaling out.
1.3	Snowflake Editions and First Setup	Which edition and cloud do I pick, and how do I log in?	Sign up for the free trial and navigate Snowsight.
1.4	SnowSQL CLI	How do I automate Snowflake from a script?	Install SnowSQL, configure named connections, run files non-interactively.
1.5	The Object Hierarchy	What contains what in Snowflake?	Walk the ACCOUNT -> DATABASE -> SCHEMA tree; set explicit session context.

sql — Chapter 1 in a single snapshot - everything you will build by the end of the chapter

```sql
-- If you can read and understand every line below by the end of Chapter 1,
-- you are ready for Chapter 2.

USE ROLE sysadmin;
USE WAREHOUSE compute_wh;

CREATE DATABASE IF NOT EXISTS mango_mart;
CREATE SCHEMA   IF NOT EXISTS mango_mart.retail;
USE DATABASE mango_mart; USE SCHEMA retail;

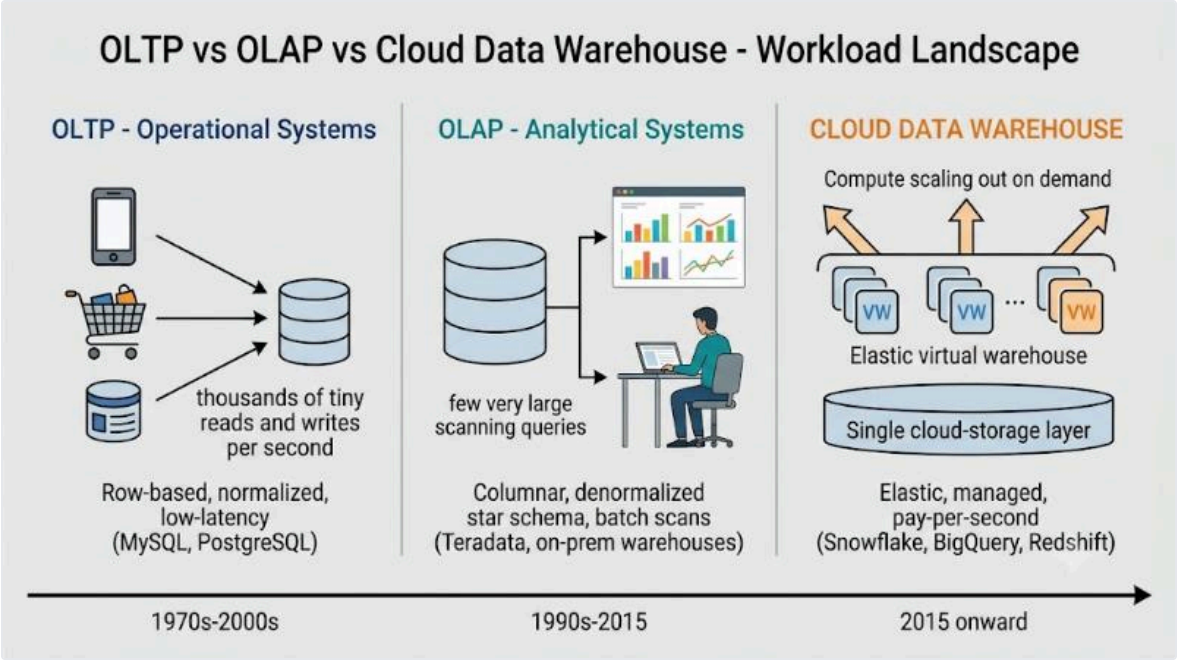
CREATE OR REPLACE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE='MEDIUM' AUTO_SUSPEND=300 AUTO_RESUME=TRUE
  MIN_CLUSTER_COUNT=1 MAX_CLUSTER_COUNT=4;

SELECT current_role(),  current_warehouse(),
```

```
current_database(), current_schema(),
current_region();
```

WHAT IS A CLOUD DATA WAREHOUSE?

Claude • AI Assistant



OLTP vs OLAP vs Cloud Data Warehouse - Workload Landscape

sql — A canonical OLTP-style query - cheap on MySQL, wasteful on Snowflake

```sql
-- OLTP: look up ONE customer's most recent order.
-- This is what your college project's MySQL does thousands of times per
second.
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = 91234
ORDER BY order_date DESC
LIMIT 1;
```

sql — A canonical OLAP-style query - the workload cloud warehouses are built for

```
-- OLAP: which product categories drove the most revenue
-- across every store in the last three years, month by month?
-- Scans hundreds of millions of rows.
SELECT DATE_TRUNC('MONTH', f.sale_date) AS sale_month,
       p.category,
       SUM(f.amount) AS total_revenue,
       COUNT(DISTINCT f.customer_id) AS unique_buyers
FROM fact_sales f
JOIN dim_product p ON p.product_id = f.product_id
WHERE f.sale_date >= DATEADD(YEAR, -3, CURRENT_DATE())
GROUP BY 1, 2
ORDER BY 1, 3 DESC;
```

Student

Sir, before we even touch Snowflake, I have a very basic question. I have used **MySQL** and **PostgreSQL** for my college projects. Why do we suddenly need something called a **cloud data warehouse**? Isn't a database enough?

Claude • AI Assistant

That is the perfect question to start with, and I want you to hold onto that confusion, because unlearning is half the battle here. The database you used for your college project - MySQL, PostgreSQL, SQL Server - is what we call an **OLTP** database.

OLTP stands for **Online Transaction Processing**. It is designed to do one thing extremely well: handle thousands of tiny, quick reads and writes per second.

Think of your college library system - a student borrows a book, the system inserts one row, updates one status, and commits. That is a transaction.

A data warehouse is a completely different animal. It is built for **OLAP** - **Online Analytical Processing**.

Instead of thousands of tiny transactions, it processes a small number of enormous queries that scan millions or billions of rows and answer questions like *'What were the top 10 best-selling categories across every store in India for the last three years, broken down by month?*'

Student

So OLTP is for the application, and OLAP is for the analyst?

Claude • AI Assistant

CONCEPT CARD

Exactly. Let me formalize that.

OLTP systems are built for **operational workloads** - booking a ticket, placing an order, updating a profile. They favour **normalized schemas** (many small tables joined together) and **row-based storage** because you usually read a small number of complete rows at a time.

OLAP systems, or data warehouses, are built for **analytical workloads** - reports, dashboards, forecasting, machine-learning feature engineering. They favour **denormalized schemas** (star or snowflake), **columnar storage** (which lets you read only the columns you need across billions of rows), and **massive parallel processing (MPP)** across many machines.

You have already seen the star schema in the Data Modeling week when we built `FACT_SALES` sitting in the middle of `DIM_CUSTOMER`, `DIM_PRODUCT`, `DIM_STORE`, and `DIM_DATE`. That is a warehouse pattern.

Student

Fine - so a warehouse is a bigger database made for analytics. Then what does 'cloud' add? Can't I just install Oracle Exadata on a server in my company's rack?

Claude • AI Assistant

ANALOGY

You can, and companies did this for decades - that world is called **on-premises** or 'on-prem'. But it comes with a very specific kind of pain.

Think of an on-prem warehouse like buying a **10-seater bus for your family**. It works - but 6 days a week, 9 seats sit empty.

One day a year the whole extended family visits and 12 people show up, and now you are short on seats. You cannot shrink the bus, and you cannot suddenly grow it.

You paid up front for the peak. That is on-prem data warehousing in one image.

REAL WORLD APPLICATION

At a large Indian bank I consulted for, the finance team ran their month-end reports on a Teradata cluster that was sized for the last week of the quarter. For the other three weeks it sat at 15% utilization.

Millions of rupees of hardware, idle. And when the auditors demanded a new report that required scanning ten years of transactions, the DBAs said *'that will freeze the cluster for eight hours.'* Nobody was happy.

That gap - between what you have provisioned and what you actually need at any given moment - is exactly what cloud data warehouses were built to close.

Student

I see. So the cloud lets you resize on demand?

Claude • AI Assistant

More than that. A modern **cloud data warehouse** gives you **elastic compute** (spin up more processing power in seconds and shut it down when you are done), **effectively unlimited storage** (petabytes on demand, no capacity planning), and **fully managed operations** (no patching, no backups to babysit, no hardware failures to handle).

You pay only for what you use - storage per terabyte per month, compute per second. Snowflake was the first product built from the ground up on this model.

AWS Redshift, Google BigQuery, Databricks SQL, and Azure Synapse are all in the same category, each with their own architectural choices.

Student

How did on-prem warehouses actually get set up? I want to understand what changed.

Claude • AI Assistant

EXAMPLE CARD

Picture the traditional on-prem setup. A team of DBAs orders 20 physical servers - call them nodes.

They rack them in a datacentre, install the warehouse software (Teradata, Netezza, Vertica), configure storage arrays, set up backup jobs, and finally load data. This is a six-month project.

Once running, adding a 21st node to handle a bigger workload takes another three months. And that entire cluster is a single, static resource - every team shares it and fights for slots.

Now picture Snowflake. You open a browser, sign up for a free trial, and in under five minutes you are running SQL against a real distributed warehouse.

You do not know or care how many machines are underneath. When you want more compute you type `ALTER WAREHOUSE MY_WH SET WAREHOUSE_SIZE = 'LARGE';` and the resize happens in a few seconds.

That single before-and-after is why cloud data warehouses have taken over the market.

Student

Where does Snowflake sit in the 'modern data stack' I keep hearing about?

Claude • AI Assistant

CONCEPT CARD

The **modern data stack** is a loose term for the set of tools most data teams use in 2026. It has five layers.

- **Source systems** - application databases (Postgres, MySQL), SaaS tools (Salesforce, Stripe), log streams (Kafka), files in cloud storage.
- **Ingestion** - tools like Fivetran, Airbyte, and Snowflake's own **Snowpipe** move data from sources into the warehouse.
- **Storage and compute** - the cloud data warehouse itself. This is where Snowflake lives.
- **Transformation** - tools like **dbt** run SQL inside the warehouse to shape raw data into clean analytics tables.
- **Consumption** - BI dashboards (Tableau, Looker, Power BI), notebooks, ML pipelines, and reverse-ETL tools.

Snowflake sits at the **center of gravity**. Every other layer eventually touches it. That is why learning it is such a high-leverage skill - the same knowledge lets you plug into ingestion, transformation, analytics, and ML work at almost any modern company.

Student

You said Snowflake was the first built for this model. What did that first insight look like?

Claude • AI Assistant

The founders of Snowflake - Benoit Dageville and Thierry Cruanes - noticed that every existing warehouse tightly coupled **compute** (the CPUs doing the work) and **storage** (the disks holding the data). That coupling forced every scaling decision to be a compromise.

Need more compute? You had to add storage too.

Need more storage? You had to pay for more compute.

Their bet was: what if we **decouple the two completely**? Store the data once in cloud object storage (S3, Azure Blob, GCS).

Let compute clusters - they called them **virtual warehouses** - spin up on demand, read from that shared storage, do their work, and shut down. That single architectural bet is the reason we are having this conversation.

We will unpack it in detail in the next subsection.

Student

One more thing - the phrase 'cloud data warehouse' feels overloaded. Are there sub-categories?

Claude • AI Assistant

CONCEPT CARD

Yes, three broad camps have emerged.

- **Cloud data warehouses** (Snowflake, BigQuery, Redshift) - SQL-first, optimized for structured and semi-structured data, best for BI and reporting.
- **Data lakes** (raw S3, ADLS, GCS with query engines like Athena) - cheap object storage, any format, best for archival and data science exploration.
- **Lakehouses** (Databricks, Snowflake's Iceberg tables) - a merger of the two, offering warehouse-like ACID guarantees on lake-like open storage formats.

Snowflake started firmly in camp one but has been steadily annexing camps two and three with features like **external tables**, **Iceberg tables**, and **Snowpark**. By the end of this refcard you will see how one platform now covers all three.

⚠ WARNING

Do not confuse a data warehouse with a **transactional database**. Never run your live e-commerce checkout on Snowflake - it is not tuned for millisecond, single-row inserts. Snowflake is for **analytics on top of** what your OLTP systems produce.

COMMON PITFALL / MISTAKE

Students often say '*Snowflake is faster than MySQL.*' That is a category error. Snowflake is not faster at booking an airline seat - MySQL is.

Snowflake is faster at scanning three billion airline booking rows to find which routes lost the most revenue last quarter. Pick the tool for the workload.

EXAM TIP CARD

SnowPro Core loves to test the OLTP/OLAP distinction and the workload types Snowflake targets. Memorize this: *Snowflake is a cloud-native, OLAP, MPP, columnar, fully managed data platform that separates storage from compute.* Every one of those adjectives is a potential exam question.

PRACTICE QUIZ

Which of the following workloads is a **poor fit** for a cloud data warehouse like Snowflake?

A) A daily job that aggregates 500 million orders into a sales dashboard

B) A recommendation engine reading five years of clickstream data

C) A payment gateway inserting 20,000 transactions per second in real time

D) A monthly customer-churn report joining CRM, billing, and support tickets

▼ Reveal Correct Answer

Answer: C. Payment gateways need low-latency single-row inserts - a classic OLTP workload. A/B/D are all analytical scans over historical data - the sweet spot for Snowflake.

EXERCISE

Take five minutes and list, for your college project, which parts would fit a cloud data warehouse and which would stay in MySQL. Hint: the login flow and cart update belong in MySQL; the '*top-selling products by category*' dashboard belongs in Snowflake.

AI DEEP DIVE PROMPT

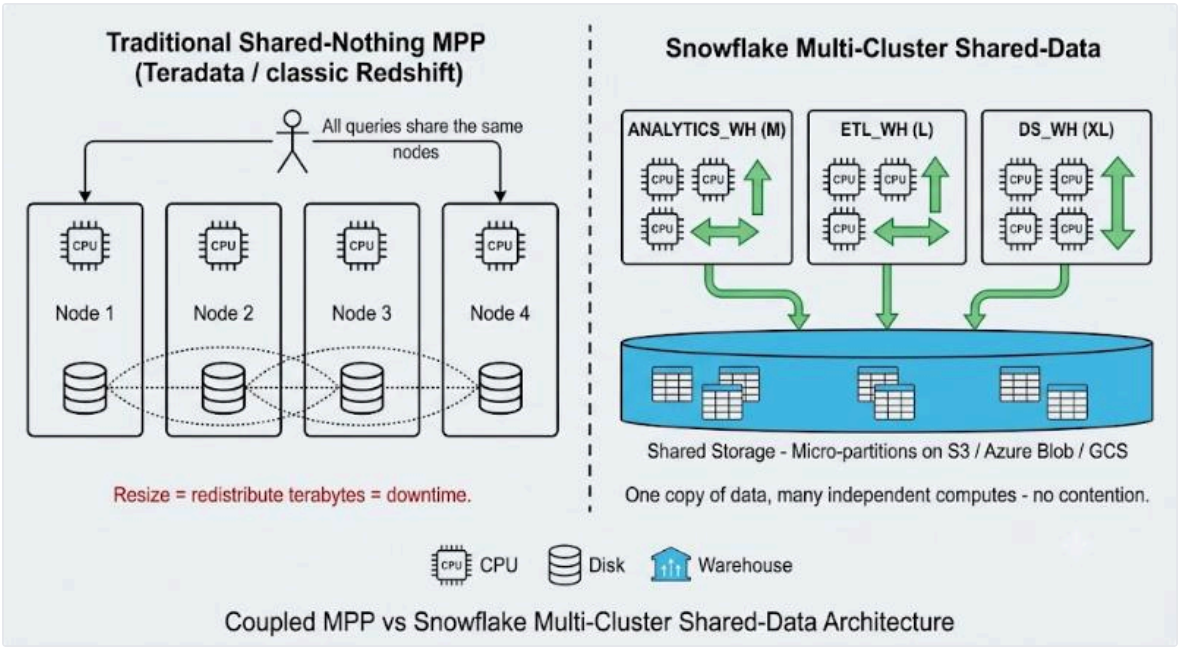
Ask an AI: '*Explain the difference between OLTP and OLAP using a coffee-shop example, and then explain where a cloud data warehouse fits.*' Compare its answer with what we just discussed and note the differences.

SUMMARY CARD

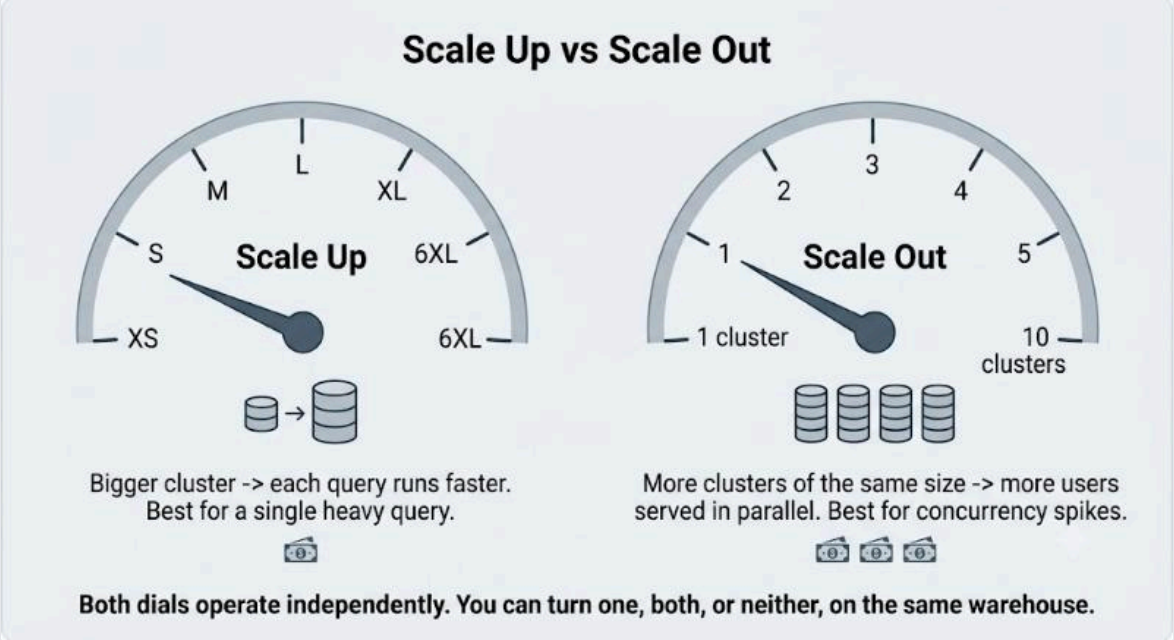
> A **cloud data warehouse** is a fully managed, elastically scalable, columnar OLAP platform designed for analytical queries over very large datasets. It replaces the fixed-capacity, high-maintenance world of on-prem warehousing with pay-as-you-go compute and effectively unlimited storage.
>
> Snowflake pioneered the model by decoupling storage from compute - the architectural decision we will study next.

WHY SNOWFLAKE - SEPARATION OF STORAGE AND COMPUTE

Claude • AI Assistant



Coupled MPP vs Snowflake Multi-Cluster Shared-Data Architecture



Scale Up vs Scale Out - Two Independent Dials

sql — Create three independent warehouses for three teams - demonstrating the decoupling

```sql
-- Analytics team: medium warehouse, auto-suspend after 5 minutes idle.
CREATE OR REPLACE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  AUTO_SUSPEND = 300
  AUTO_RESUME = TRUE
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 4       -- scale OUT for concurrency
  SCALING_POLICY = 'STANDARD'
  COMMENT = 'BI dashboards for the analytics team.';

-- Data engineering: large warehouse for heavy nightly ETL.
CREATE OR REPLACE WAREHOUSE etl_wh
  WAREHOUSE_SIZE = 'LARGE'    -- scaled UP for heavy jobs
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  COMMENT = 'Nightly ELT loads.';

-- Data science: XL warehouse for training, only when needed.
CREATE OR REPLACE WAREHOUSE ds_wh
  WAREHOUSE_SIZE = 'XLARGE'
  AUTO_SUSPEND = 600
  AUTO_RESUME = TRUE
  COMMENT = 'Ad-hoc ML feature engineering.';
```

sql — Scale up and scale out on the fly - no downtime

```
-- Scale UP: temporarily upgrade for a heavy month-end job.
ALTER WAREHOUSE etl_wh SET WAREHOUSE_SIZE = 'XLARGE';

-- ...run the heavy job...

-- Scale back down.
ALTER WAREHOUSE etl_wh SET WAREHOUSE_SIZE = 'LARGE';

-- Scale OUT: allow the analytics warehouse to burst to 6 clusters
-- for Monday morning dashboard load.
ALTER WAREHOUSE analytics_wh SET MAX_CLUSTER_COUNT = 6;
```

Student

You dropped a big hint at the end of the last subsection - 'decouple storage from compute'. That sounds important but abstract. Can we open it up?

Claude • AI Assistant

It is the single most important idea in this entire refcard, so we are going to spend real time on it. In every warehouse before Snowflake - Teradata, Netezza, even AWS Redshift in its original form - **storage and compute lived on the same machines**.

Each node in the cluster had its own local disks; the data was partitioned across those disks; and the CPUs on that node were the only ones who could read that data. This design is called **shared-nothing MPP** (massively parallel processing), and it was the state of the art for two decades.

It also created three specific pains.

Student

Which three?

Claude • AI Assistant

CONCEPT CARD

The three pains of coupled storage and compute:

1. **Rigid scaling.** If your dataset grows but your workload stays the same, you still have to add nodes - and thus more CPU - because you cannot add disk without adding CPU. You pay for compute you do not need.
2. **Workload contention.** Every user's query competes for the same CPUs. A data scientist's giant training query blocks the finance team's daily dashboard.
3. **Expensive resizing.** Because data is physically partitioned across nodes, adding or removing a node forces a **redistribution** of terabytes of data. This can take hours and disrupt the cluster.

Student

And Snowflake's answer to all three is decoupling?

Claude • AI Assistant

Yes. Snowflake stores every table exactly once in a **shared, cloud-object-storage layer** - AWS S3, Azure Blob Storage, or Google Cloud Storage, depending on your region.

That layer is a single logical copy. It does not care who is reading.

Compute, meanwhile, is delivered through **virtual warehouses** - independent, sized clusters of compute nodes that you spin up when needed and shut down when done. Two teams can run two different virtual warehouses against the same table at the same time, and neither slows the other down.

Storage scales on its own. Compute scales on its own.

And you pay for each separately.

Student

Wait - every warehouse reads from S3? Isn't S3 slow?

Claude • AI Assistant

COMMON PITFALL / MISTAKE

Great catch, and this is where students often reject the idea prematurely. Reading raw S3 is slow - but Snowflake does not read raw S3 per query.

Each virtual warehouse has a **local SSD cache** and a **result cache**. The first time a warehouse scans a table, it pulls the needed micro-partitions from S3 and caches them locally.

Subsequent queries against the same data hit the SSD cache at RAM-adjacent speeds. The result cache goes one step further - if you re-run an identical query and the underlying data has not changed, Snowflake returns the previous result from cache in **milliseconds**, without spinning up compute at all.

That layered caching is why the decoupled model performs competitively with coupled MPP systems.

Student

How is the architecture organized as layers?

Claude • AI Assistant

CONCEPT CARD

Snowflake's architecture has **three layers**, stacked bottom to top:

- **Storage layer** - the single source of truth. Data lives as compressed, columnar **micro-partitions** in cloud object storage. Immutable, versioned, replicated across availability zones by the cloud provider.
- **Query processing layer** - a pool of **virtual warehouses**, each an independent MPP cluster of compute nodes. This is where SQL actually runs.
- **Cloud services layer** - the brain: authentication, metadata management, query parsing and optimization, transaction coordination, security. Shared across every user and warehouse.

We will unpack each layer in detail in Chapter 2. For now, keep this picture in mind: **one shared storage, many independent computes, one central brain**.

Student

Can you give me a concrete scenario where the decoupling actually saves money or time?

Claude • AI Assistant

EXAMPLE CARD

Consider **Mango Mart**, the retail company we used in the Data Modeling week. Mango Mart has three teams:

- **Analytics team** - runs daily dashboards from 8 AM to 8 PM.
- **Data engineering team** - runs nightly ETL loads from midnight to 4 AM.
- **Data science team** - runs sporadic ML training jobs during the day.

On an old on-prem warehouse, all three fight for the same cluster. The ETL job slows down the morning dashboards. The training job kills the ETL window. Everyone is unhappy.

On Snowflake, Mango Mart creates **three separate virtual warehouses** - `ANALYTICS_WH` (Medium, auto-suspend 5 minutes), `ETL_WH` (Large, runs midnight-4 AM), and `DS_WH` (XLarge, auto-suspend 10 minutes). All three read the same tables.

None slows down the others. Costs stay bounded because each warehouse suspends when idle.

This scenario - three isolated compute pools over one shared dataset - is impossible in the coupled model. In Snowflake it is a five-minute setup.

Student

You said 'multi-cluster shared-data architecture'. Is that a fourth thing?

Claude • AI Assistant

CONCEPT CARD

It is the name Snowflake uses for the whole pattern I just described. **Multi-cluster** means many independent compute clusters can run at once.

Shared-data means they all read from the same storage layer. You will see this exact phrase - 'multi-cluster, shared-data' - in every Snowflake marketing deck and every SnowPro exam.

Own it.

Student

Are there any downsides to the model?

Claude • AI Assistant

⚠ WARNING

Two honest ones.

- **Cold-query latency.** The very first query against a table on a freshly resumed warehouse must pull data from remote storage into local cache. It is a bit slower than a warm query. Snowflake mitigates this with pre-warming and result caching, but it exists.
- **Cost of enthusiasm.** Because spinning up a warehouse is so easy, teams sometimes leave many warehouses running or oversize them 'just in case'. Snowflake charges per second, so casual mistakes become real invoices. **Cost governance is now an operational skill.** We will devote a full subsection to it in Chapter 5.

Student

What about concurrency? What if 200 analysts hit the dashboard at 9 AM sharp?

Claude • AI Assistant

This is where the **multi-cluster warehouse** feature shines. A single Snowflake virtual warehouse can be configured to auto-scale from, say, one cluster up to five clusters as query load rises, and shrink back as it falls.

Each cluster handles a share of the queries independently. From the user's point of view they are all querying `ANALYTICS_WH` - Snowflake transparently routes them to whichever cluster has capacity.

This is a completely different mechanism from resizing a single warehouse (which we call *scaling up*) - this is called *scaling out*, and it is designed for **concurrency spikes** rather than heavy single queries.

Student

So scaling up handles bigger queries, scaling out handles more queries?

Claude • AI Assistant

CONCEPT CARD

Precisely. Memorize the two-word distinction:

- **Scale up** - make each cluster bigger (XS -> S -> M -> L -> XL -> 2XL ... 6XL). Helps a **single heavy query** finish faster.
- **Scale out** - add more clusters of the same size. Helps **many concurrent queries** each finish on time.

Both dials exist independently on every warehouse. You will tune them constantly in real projects.

ANALOGY

Think of a restaurant kitchen. **Scaling up** is buying a bigger oven so one wedding cake bakes faster. **Scaling out** is hiring more chefs so a busy Saturday night with 200 orders still gets served on time. Same restaurant, two different problems, two different solutions.

REAL WORLD APPLICATION

I once worked with a Bangalore fintech that ran their entire nightly risk-model batch on Snowflake. The batch used a `LARGE` warehouse (scale up - 8 nodes) for two hours a night.

Daytime BI dashboards ran on a separate `MEDIUM` warehouse with **multi-cluster set to auto-scale 1-4 clusters** (scale out). One is optimized for throughput on a heavy job; the other for concurrency across many small queries.

The teams never fought for compute again, and their monthly bill dropped 40% because both warehouses auto-suspended when idle. That is decoupling paying real dividends.

COMMON PITFALL / MISTAKE

Students see 'auto-scale' and think Snowflake automatically picks the warehouse size for them. **It does not.** You still pick the T-shirt size (up-scaling is a manual `ALTER WAREHOUSE` command).

Only the *number of clusters* auto-scales, and only if you configured `MIN_CLUSTER_COUNT` and `MAX_CLUSTER_COUNT` explicitly. Nothing is fully hands-off.

EXAM TIP CARD

SnowPro Core almost always includes a question distinguishing scaling up from scaling out. Rehearse the exact phrasing: '*Scaling up adds compute to a single cluster to speed a heavy query. Scaling out adds more clusters of the same size to handle more concurrent queries.*'

PRACTICE QUIZ

Two teams complain: Team A's daily report takes 90 minutes and is too slow. Team B's dashboard is fast per user but times out when 150 users log in at 9 AM. What do you do for each?

A) Scale up Team A's warehouse; scale out Team B's warehouse

B) Scale out Team A's warehouse; scale up Team B's warehouse

C) Scale up both

D) Scale out both

▼ Reveal Correct Answer

Answer: A. Team A has a heavy single query - scale up. Team B has concurrency - scale out.

EXERCISE

For each of these three workloads, decide whether you would scale up, scale out, or both:

1. A quarterly financial close that scans 5 years of ledger data - heavy but rare.
2. A public marketing dashboard used by 500 people at 9 AM.
3. A live product-catalog analytics tool used by internal merchandisers throughout the day, with occasional deep-dive queries.

(Suggested answers: 1 = scale up; 2 = scale out; 3 = both - moderate scale-up plus multi-cluster scale-out.)

AI DEEP DIVE PROMPT

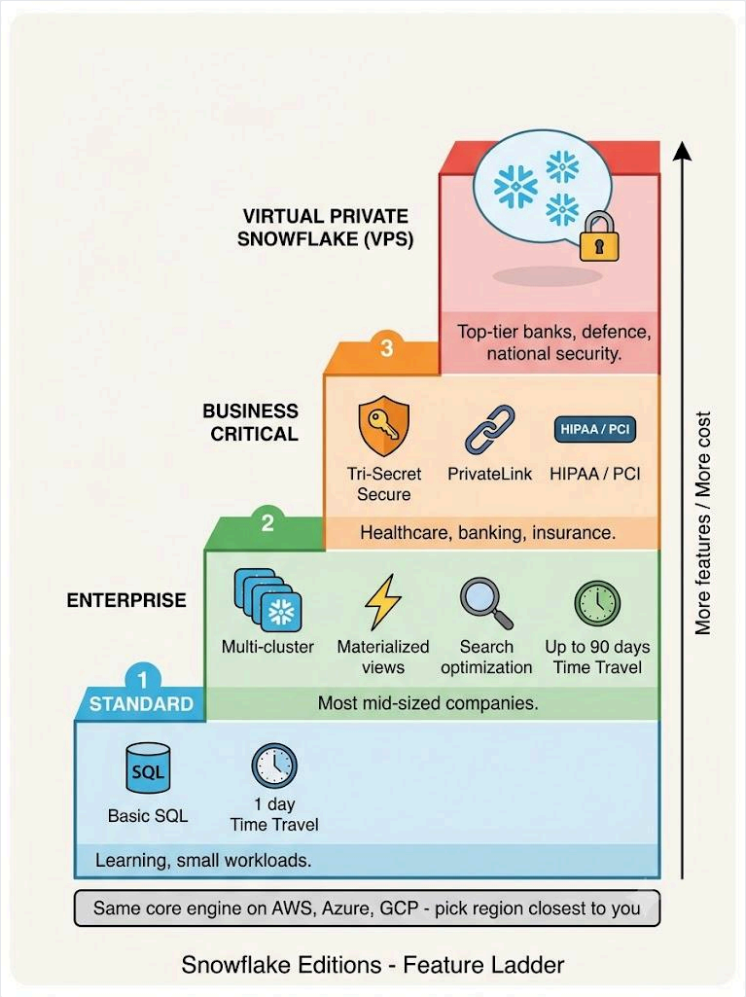
Ask an AI: '*Explain the difference between shared-nothing MPP and Snowflake's multi-cluster shared-data architecture using a warehouse-loading analogy.*' Then critique the answer against what we discussed.

SUMMARY CARD

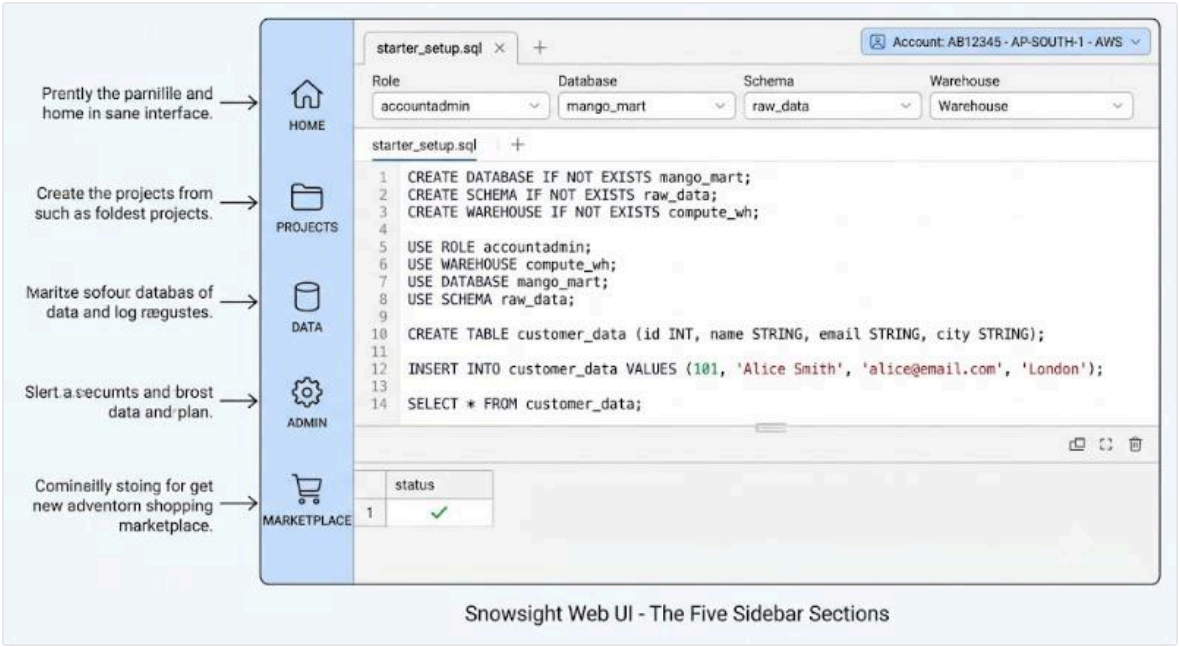
Snowflake decouples storage from compute by keeping one canonical copy of your data in cloud object storage, and letting you spin up any number of independent **virtual warehouses** to query it. This eliminates workload contention, allows each team to right-size its own compute, and enables both **scaling up** (bigger clusters for heavy queries) and **scaling out** (more clusters for more concurrent queries) - the ideas the rest of the refcard depends on.

SNOWFLAKE EDITIONS AND FIRST SETUP

Claude • AI Assistant



Snowflake Editions - Feature Ladder



Snowsight Web UI - The Five Sidebar Sections

sql — First-run starter setup - creates the shared home base for the rest of the refcard

```sql
-- Run these in a Snowsight worksheet after your first login.
-- Role: SYSADMIN. Warehouse: COMPUTE_WH (created for you).

CREATE DATABASE IF NOT EXISTS mango_mart
  COMMENT = 'Practice retail database for the RC-906-2026 refcard.';

CREATE SCHEMA IF NOT EXISTS mango_mart.retail
  COMMENT = 'Star-schema and semi-structured playground.';

-- Activate the context so subsequent statements know where to run.
USE DATABASE mango_mart;
USE SCHEMA   retail;

-- Quick sanity check.
SELECT CURRENT_DATABASE(), CURRENT_SCHEMA(), CURRENT_WAREHOUSE(),
       CURRENT_ROLE(),     CURRENT_REGION();
```

sql — Explore the two default databases every account receives

```sql
-- What sample tables ship with every account?
SHOW SCHEMAS IN DATABASE snowflake_sample_data;
SHOW TABLES  IN SCHEMA  snowflake_sample_data.tpch_sf1;

-- Meta-question: which queries did I run in the last day?
-- (Runs against ACCOUNT_USAGE - remember: up to 3 hours latency.)
SELECT query_id, user_name, warehouse_name, execution_status,
       total_elapsed_time / 1000 AS seconds
FROM   snowflake.account_usage.query_history
WHERE  start_time > DATEADD('DAY', -1, CURRENT_TIMESTAMP())
ORDER BY start_time DESC
LIMIT 20;
```

Student

Alright, I am sold on the architecture. How do I actually get into Snowflake? Do I have to pay?

Claude • AI Assistant

Not a rupee to get started. Snowflake offers a **30-day free trial** with **US$400 of credits** - enough for you to build a real project, run large-ish queries, and even experiment with Snowpark and Cortex.

Every exercise in this refcard is designed to fit inside that free trial. Before you sign up, though, you need to understand one thing: **which edition to pick**.

Snowflake sells the same core engine at four price tiers, and picking the wrong one for a real project can cost your company thousands of dollars a month.

Student

What are the editions?

Claude • AI Assistant

CONCEPT CARD

Snowflake has **four editions** - Standard, Enterprise, Business Critical, and Virtual Private Snowflake (VPS). Each is a superset of the previous one.

- **Standard** - The base tier. All core SQL, all data types, up to **1 day of Time Travel**, standard support. Perfect for learning and small workloads.
- **Enterprise** - Everything in Standard, plus up to **90 days of Time Travel**, **multi-cluster warehouses** (scale out), **materialized views**, **search optimization service**, and **column-level security**. This is what most mid-sized companies pick.
- **Business Critical** - Everything in Enterprise, plus **HIPAA / PCI DSS** compliance, **customer-managed encryption keys (Tri-Secret Secure)**, **failover across regions**, and **private connectivity** (AWS PrivateLink, Azure Private Link). Chosen by healthcare, banking, insurance.
- **Virtual Private Snowflake (VPS)** - Everything in Business Critical, running in a **completely isolated Snowflake environment** with no shared metadata layer. Used only by the most security-sensitive customers, typically top-tier banks and defence contractors.

Student

For learning, Standard is fine?

Claude • AI Assistant

> ⚠ **WARNING**
>
> For the trial itself, Standard is fine - but note that **multi-cluster warehouses** and **materialized views** need Enterprise. When you sign up for the trial you can choose which edition you want to try.
>
> **Pick Enterprise for this refcard's exercises** - the free credits work the same, and you unlock the multi-cluster feature we discussed in the previous subsection. If you pick Standard, some of the later hands-on labs (Chapters 3 and 4) will error out.

Student

What about cloud provider? Which one should I choose?

Claude • AI Assistant

Snowflake runs the exact same product on **AWS**, **Azure**, and **Google Cloud Platform**. The choice of cloud provider mostly affects **regional availability**, **network latency to your existing systems**, and **pricing**, but not features.

For an Indian student experimenting on the free trial, either **AWS Asia Pacific (Mumbai)** or **Azure Central India** gives you the lowest latency. If your college or company already uses one cloud, pick that same cloud so data transfer between your existing systems and Snowflake is free and fast.

For the exercises in this refcard, any region works - the trial is not sensitive to your choice. Just remember which region you picked because you cannot move a Snowflake account between regions later without a full data migration.

Student

Okay, let me imagine I have signed up. What do I see?

Claude • AI Assistant

> **EXAMPLE CARD**
>
> After signup you receive a URL that looks like `https://ab12345.ap-south-1.aws.snowflakecomputing.com`. That whole string is your **account identifier** - memorize its structure:
>
> - `ab12345` is your Snowflake **account locator** (unique per account).

- `ap-south-1` is the cloud **region**.
- `aws` is the cloud **provider**.
- The trailing `snowflakecomputing.com` is the base domain.

You will need this identifier every time you configure SnowSQL, Snowpark, dbt, or any BI tool. Save it somewhere safe.

When you open the URL, the browser loads **Snowsight**, Snowflake's modern web interface. Snowsight has five main sections in its left sidebar:

1. **Home** - a dashboard of recent work and shortcuts.
2. **Projects** - worksheets (SQL notebooks), dashboards, and Snowpark notebooks.
3. **Data** - databases, schemas, tables, and views (the object browser).
4. **Admin** - warehouses, users, roles, resource monitors, cost.
5. **Marketplace** - shared datasets you can subscribe to instantly.

Student

What was there before Snowsight?

Claude • AI Assistant

The older interface is called the **Classic Console**. You may still see references to it in older tutorials on the internet.

It is being retired - always use **Snowsight** for anything new. If a screenshot in a tutorial looks like an old, dark-themed grid of boxes, that is Classic.

If it looks like a modern web app with a sidebar and rich charts, that is Snowsight.

Student

What is a **worksheet**?

Claude • AI Assistant

CONCEPT CARD

A **worksheet** is Snowsight's core interface for running SQL. Think of it as a persistent, versioned SQL notebook.

You pick a role, database, schema, and warehouse from dropdowns at the top; type SQL in the main pane; press `Ctrl+Enter` (or `Cmd+Enter` on macOS) to run the highlighted

statement; and see results in a grid at the bottom that supports sorting, filtering, and one-click charting. Every worksheet is saved automatically, versioned, and shareable with teammates.

You will do most of your day-to-day Snowflake work in worksheets during learning; in production you graduate to Snowpark notebooks, dbt, and the SnowSQL CLI (which we cover next).

Student

What is the very first thing I should run when I log in?

Claude · AI Assistant

EXAMPLE CARD

A tradition. Open a worksheet, pick role `SYSADMIN`, warehouse `COMPUTE_WH` (Snowflake creates one automatically on account creation), and run these three statements - they establish the classic starter setup for a retail practice project you can extend for the rest of this refcard:

sql — Code Example

```
CREATE DATABASE IF NOT EXISTS mango_mart;
CREATE SCHEMA  IF NOT EXISTS mango_mart.retail;
USE DATABASE mango_mart; USE SCHEMA retail;
```

Now you have a database, a schema, and an active context. You are ready to create tables, load data, and run queries. Everything you do in the exercises through the rest of this refcard will live inside `mango_mart.retail` unless we say otherwise.

Student

What are those two default databases I see - `SNOWFLAKE` and `SNOWFLAKE_SAMPLE_DATA` ?

Claude · AI Assistant

CONCEPT CARD

Both are gifts from Snowflake, and both are useful.

- **`SNOWFLAKE`** - a **shared read-only database** containing views over Snowflake's internal metadata. When you want to know 'who ran what query yesterday' or 'how many credits did each warehouse consume last month', you query views inside `SNOWFLAKE.ACCOUNT_USAGE`. This is how every FinOps and audit workflow on Snowflake is built.
- **`SNOWFLAKE_SAMPLE_DATA`** - TPC-DS and TPC-H benchmark datasets shared with every account, with tables ranging from a few thousand rows to a few billion. Perfect for practicing joins and aggregations at real scale without loading anything yourself. In this refcard we will occasionally point at it to demonstrate query patterns before we load our own retail data.

⚠ WARNING

The `ACCOUNT_USAGE` views have a **latency of up to 3 hours**. If a query you just ran does not appear in `QUERY_HISTORY`, wait - do not assume it was lost.

There is a lower-latency alternative called `INFORMATION_SCHEMA`, scoped to the current database, that we will use in Chapter 3.

COMMON PITFALL / MISTAKE

Students on the free trial often forget the trial is **time-boxed** (30 days) and **credit-boxed** (US$400). They leave a warehouse running overnight, come back the next day, and find their trial exhausted.

Every warehouse you create in the exercises has `AUTO_SUSPEND` set for a reason - never disable it, and always double-check with `SHOW WAREHOUSES;` before you close the browser.

ANALOGY

Choosing a Snowflake edition is like choosing an airline ticket class. Standard is economy - you get from A to B.

Enterprise is business class - extra legroom (materialized views, multi-cluster) and a proper meal (search optimization). Business Critical is first class - full compliance suite, private connectivity.

VPS is a chartered jet - completely isolated for exceptional cases.

REAL WORLD APPLICATION

A student I mentored built her final-year project on the free trial in exactly 27 days. She loaded 200 GB of open Kaggle retail data, built a star schema, ran BI dashboards from Metabase, and even trained a Cortex-based summarization query.

Total credits used: $180 of the $400. **The free trial is not a toy - it is a real production platform you happen to be renting for free.** Use it seriously.

EXAM TIP CARD

SnowPro Core loves edition-feature mapping questions. Memorize the *first-appearance edition* of the top features:

- **Time Travel 1 day** - Standard.
- **Time Travel up to 90 days** - Enterprise.
- **Multi-cluster warehouses** - Enterprise.
- **Materialized views** - Enterprise.
- **Search optimization** - Enterprise.
- **Dynamic data masking** - Enterprise.
- **Tri-Secret Secure customer-managed keys** - Business Critical.
- **PrivateLink** - Business Critical.
- **HIPAA / PCI compliance** - Business Critical.

PRACTICE QUIZ

A healthcare startup wants to store patient records in Snowflake. Which edition **at minimum** must they choose?

A) Standard B) Enterprise C) Business Critical D) VPS

▼ Reveal Correct Answer

Answer: C. HIPAA compliance is a Business Critical feature.

EXERCISE

Sign up for the Snowflake free trial (Enterprise edition, AWS Mumbai or Azure Central India). Log in, open a worksheet, and run the three-statement starter setup for `mango_mart.retail` .

Take a screenshot of the successful results panel - that is your proof of getting to the starting line.

AI DEEP DIVE PROMPT

Ask an AI: '*Compare Snowflake Enterprise, Business Critical, and VPS editions in terms of security, compliance, and cost.*' Cross-check its answer against the edition table above.

SUMMARY CARD

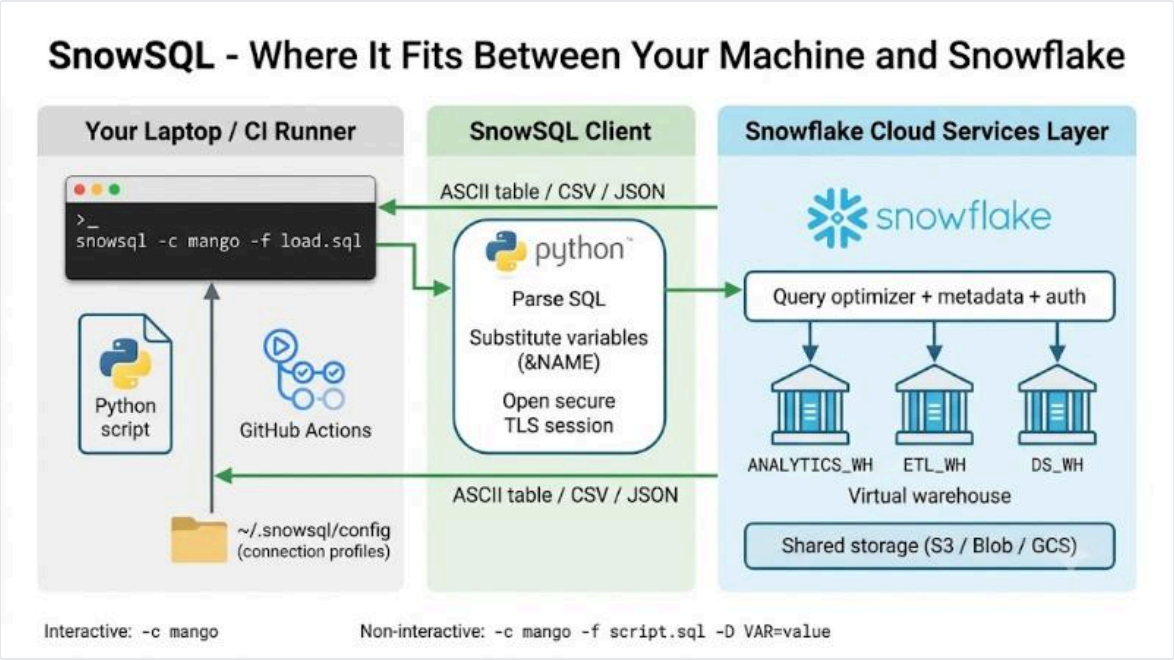
Snowflake offers a **30-day, $400 free trial** across four editions (Standard, Enterprise, Business Critical, VPS) on three clouds (AWS, Azure, GCP). Pick **Enterprise + your nearest**

region for this refcard.

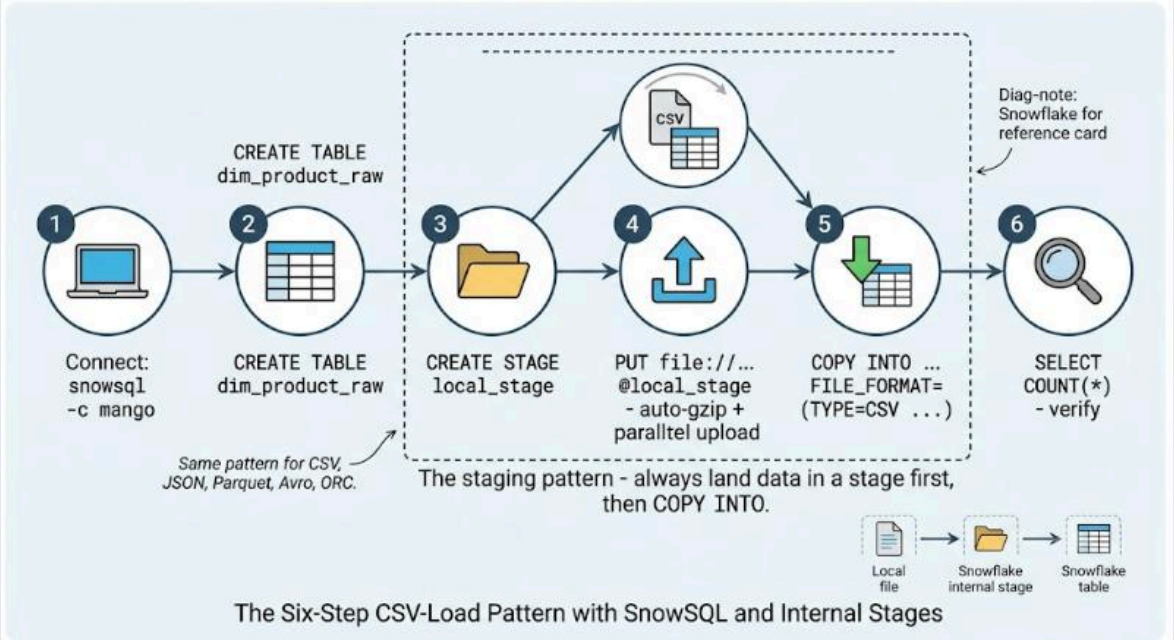
Snowsight is the modern web UI where you will run everything; the `SNOWFLAKE` and `SNOWFLAKE_SAMPLE_DATA` databases are your metadata and practice data respectively; and `mango_mart.retail` is our shared home base for the rest of the exercises.

SNOWSQL - THE COMMAND-LINE CLIENT

Claude • AI Assistant



SnowSQL - Where It Fits Between Your Machine and Snowflake



The Six-Step CSV-Load Pattern with SnowSQL and Internal Stages

bash — SnowSQL config file - reusable named connections

```
# ~/.snowsql/config

[connections.mango]
accountname   = ab12345.ap-south-1.aws
username      = SRIKANTH
# password omitted on purpose - SnowSQL will prompt
dbname        = MANGO_MART
schemaname    = RETAIL
warehousename = COMPUTE_WH
rolename      = SYSADMIN

[connections.mango_keypair]
accountname      = ab12345.ap-south-1.aws
username         = SRIKANTH_SVC
authenticator    = SNOWFLAKE_JWT
private_key_path = ~/.snowsql/rsa_key.p8
dbname           = MANGO_MART
schemaname       = RETAIL
warehousename    = ETL_WH
rolename          = ETL_ROLE

[variables]
load_date = 2026-09-19
```

bash — Running a SQL file non-interactively from a shell script - the ETL pattern

```
#!/usr/bin/env bash
# nightly_load.sh - triggered by cron at 01:00 IST
set -euo pipefail

LOAD_DATE=$(date +%Y-%m-%d)
LOG_DIR=/var/log/mango_mart
mkdir -p "$LOG_DIR"

snowsql -c mango_keypair \
  -f /opt/mango_mart/sql/load_daily_sales.sql \
  -D LOAD_DATE="$LOAD_DATE" \
  -D SOURCE_BUCKET=s3://mango-inbound/sales/ \
  -o output_format=csv \
  -o log_level=INFO \
  > "$LOG_DIR/load_${LOAD_DATE}.log" 2>&1

echo "Load for $LOAD_DATE completed - see $LOG_DIR/load_${LOAD_DATE}.log"
```

Student

Sir, Snowsight in the browser looks great. Why do we still need a **command-line client**? Isn't that a step backwards?

Claude • AI Assistant

It feels that way at first, and I want to address that head-on. The browser is wonderful for **exploration and ad-hoc work** - you write a query, see results, sort a chart. But almost every real production workflow lives outside the browser:

- A nightly job that runs 40 SQL files in a specific order.
- A CI/CD pipeline that deploys schema changes when someone merges a pull request.
- A monitoring script that emails you if last night's ETL failed.
- A one-off bulk-load of 500 GB of CSV files from your laptop.

None of these want a human clicking buttons in a browser. They want a **script-friendly command-line tool** that they can call from bash, from Python, from GitHub Actions, from Airflow. That tool is **SnowSQL**.

Student

What exactly is SnowSQL?

Claude • AI Assistant

CONCEPT CARD

SnowSQL is Snowflake's official command-line client. It is written in Python, distributed as a single installer for macOS, Linux, and Windows, and it does one thing: **connect to Snowflake and run SQL**.

You can use it interactively (like `mysql` or `psql`), non-interactively (`snowsql -f my_script.sql`), or embedded inside shell scripts and CI jobs. It supports variables, includes, output-format switching (CSV, TSV, JSON), and even a `!spool` command to capture query output to a file.

Student

How do I install it?

Claude • AI Assistant

EXAMPLE CARD

Installation is a two-minute task.

- **macOS (Homebrew)**: `brew install --cask snowflake-snowsql`
- **Linux (RPM/DEB)**: download the installer from `https://developers.snowflake.com/snowsql/` and run it.
- **Windows (MSI)**: download and double-click the MSI installer.

After installation, open a terminal and run `snowsql -v`. You should see something like `Version: 1.3.0`.

If you see 'command not found', restart your terminal so it picks up the new PATH entry, and if it still fails, add `/Applications/SnowSQL.app/Contents/MacOS/` (macOS) or `C:\Program Files\Snowflake SnowSQL\` (Windows) to your PATH manually.

Student

How do I connect it to my Snowflake account?

Claude · AI Assistant

Two ways: **command-line arguments** or a **config file**. For a single connection you can type everything on the command line:

code — Code Example

```
snowsql -a ab12345.ap-south-1.aws -u SRIKANTH -d MANGO_MART -s RETAIL -w
COMPUTE_WH -r SYSADMIN
```

The flags mean **a**ccount, **u**ser, **d**atabase, **s**chema, **w**arehouse, **r**ole. SnowSQL prompts for your password, then drops you into an interactive session at a prompt that looks like `SRIKANTH#COMPUTE_WH@MANGO_MART.RETAIL>` .

That single-line prompt encodes your entire session context, which is a very useful safety cue.

Typing all those flags every time is tedious, so almost everyone uses the **config file** approach. SnowSQL reads `~/.snowsql/config` (Linux/macOS) or `%USERPROFILE%\.snowsql\config` (Windows). Add a named connection:

code — Code Example

```
[connections.mango]
accountname = ab12345.ap-south-1.aws
username    = SRIKANTH
password    = your_password_here
dbname      = MANGO_MART
schemaname  = RETAIL
warehousename = COMPUTE_WH
rolename    = SYSADMIN
```

Now `snowsql -c mango` is enough. You can define as many named connections as you want - one for dev, one for prod, one for a client project - and switch between them with `-c` .

Student

Storing a password in a plain text file feels risky.

Claude · AI Assistant

⚠ WARNING

Excellent instinct. **Never store production passwords in the config file.** Options:

1. **Omit the password field entirely** - SnowSQL will prompt for it every session.
2. **Use key-pair authentication** - generate an RSA key, register the public key with your Snowflake user, and put `private_key_path = ~/.snowsql/rsa_key.p8` in the config. No password anywhere on disk.
3. **Use SSO** - for enterprise accounts, `authenticator = externalbrowser` triggers your organization's SSO flow.

For the exercises in this refcard, on the free trial with a throwaway account, storing the password is acceptable. **Never** do this for a company account.

Student

What does the interactive session feel like once I am in?

Claude • AI Assistant

EXAMPLE CARD

Very familiar to anyone who has used `mysql` or `psql`. You get a prompt; you type SQL terminated with `;`; you see results as an ASCII table. Some useful built-in commands (all prefixed with `!`):

- `!help` - list all built-in commands.
- `!queries` - recent queries in this session.
- `!spool /tmp/out.csv` - capture subsequent query output to a file. `!spool off` to stop.
- `!source /path/to/file.sql` - execute a SQL file from within the interactive session.
- `!set output_format=csv` - switch result format (options: `psql`, `plain`, `csv`, `tsv`, `json`).
- `!exit` or `Ctrl+D` - leave the session.

Student

How do I run a SQL file non-interactively?

Claude • AI Assistant

EXAMPLE CARD

Two flags cover 90% of the cases.

- `snowsql -c mango -f my_script.sql` - execute the entire file, print results to stdout, exit.
- `snowsql -c mango -q "SELECT current_version();"` - run a single quick query and exit.

You can also pass variables into the script from the shell:

code — Code Example

```
snowsql -c mango \
  -f load_daily_sales.sql \
  -D LOAD_DATE=2026-09-19 \
  -D SOURCE_BUCKET=s3://mango-inbound/
```

Inside the SQL file, refer to the variables as `&LOAD_DATE` and `&SOURCE_BUCKET`. This is exactly how a nightly cron or Airflow task calls SnowSQL - the same script, different dates, different sources.

Student

Give me one full end-to-end example - connect, load a file, verify, exit.

Claude • AI Assistant

EXAMPLE CARD

Here is a complete session for loading a small local CSV into a Mango Mart table:

code — Code Example

```
# 1. Enter interactive session on the mango connection.
snowsql -c mango

# 2. Inside SnowSQL - create a table and a stage.
SRIKANTH#COMPUTE_WH@MANGO_MART.RETAIL>
CREATE OR REPLACE TABLE dim_product_raw (
  product_id STRING, name STRING, category STRING, price NUMBER(10,2));

CREATE OR REPLACE STAGE local_stage;

# 3. Upload the local file into the internal stage.
PUT file:///Users/srikanth/data/products.csv @local_stage;
```

```
# 4. Copy from the stage into the table.
COPY INTO dim_product_raw
  FROM @local_stage/products.csv.gz
  FILE_FORMAT = (TYPE=CSV FIELD_OPTIONALLY_ENCLOSED_BY='"' SKIP_HEADER=1);

# 5. Verify.
SELECT COUNT(*) FROM dim_product_raw;

# 6. Exit.
!exit
```

That six-step pattern - connect, define target, stage the file, `COPY INTO`, verify, exit - is the daily bread of a data engineer using Snowflake. You will run it a thousand times in your career.

ANALOGY

Snowsight is like a graphical IDE - great for writing and debugging. SnowSQL is like `bash` - great for scripting and automation. A working data engineer uses both, choosing whichever fits the moment.

COMMON PITFALL / MISTAKE

Students often try to run interactive-only commands (like `!spool`) inside `-f` scripts and are confused when they fail. **`!` commands are SnowSQL client commands, not SQL** - they only work in interactive mode.

In non-interactive scripts, use pure SQL and shell redirection (`snowsql ... -o output_format=csv > out.csv`).

WARNING

By default, SnowSQL uses **variable substitution** (`&NAME`). If your SQL happens to contain a literal `&` (for example in a string), SnowSQL will try to interpret it. Disable substitution with `-o variable_substitution=false` when you do not need it.

REAL WORLD APPLICATION

A retail client of mine has a `deploy.sh` script that reads a folder of numbered SQL files (`01_tables.sql`, `02_views.sql`, `03_grants.sql`) and applies them in order via SnowSQL, using a **transaction wrapper** so any failure rolls back the entire deployment. That single script is their entire schema-migration system - no fancy tooling required.

SnowSQL is quietly the backbone of enormous data platforms because of exactly this composability.

EXAM TIP CARD

SnowPro Core will ask you to identify the SnowSQL config-file location, the flag meanings (`-a` , `-u` , `-d` , `-s` , `-w` , `-r` , `-f` , `-q` , `-D`), and the correct location for private keys (`private_key_path`). Practice reading a config file and predicting the connection prompt string.

PRACTICE QUIZ

Which SnowSQL invocation runs a script and passes `RUN_DATE=2026-09-19` as a variable?

A) `snowsql -c mango -q daily.sql -D RUN_DATE=2026-09-19`

B) `snowsql -c mango -f daily.sql -D RUN_DATE=2026-09-19`

C) `snowsql -c mango --file daily.sql --var RUN_DATE=2026-09-19`

D) `snowsql -c mango -s daily.sql --D RUN_DATE=2026-09-19`

▼ Reveal Correct Answer

Answer: B. `-f` runs a file; `-D` sets a variable.

EXERCISE

Install SnowSQL on your laptop, add a `[connections.mango]` block to `~/.snowsql/config` (with password prompt, not stored), connect with `snowsql -c mango` , and run `SELECT CURRENT_VERSION();` . Then write a two-line script `hello.sql` with a single `SELECT` and run it with `snowsql -c mango -f hello.sql` .

AI DEEP DIVE PROMPT

Ask an AI: '*Compare Snowflake's SnowSQL CLI to psql for PostgreSQL - what is the same, what is different, what is uniquely Snowflake?*' Compare its answer with what we discussed.

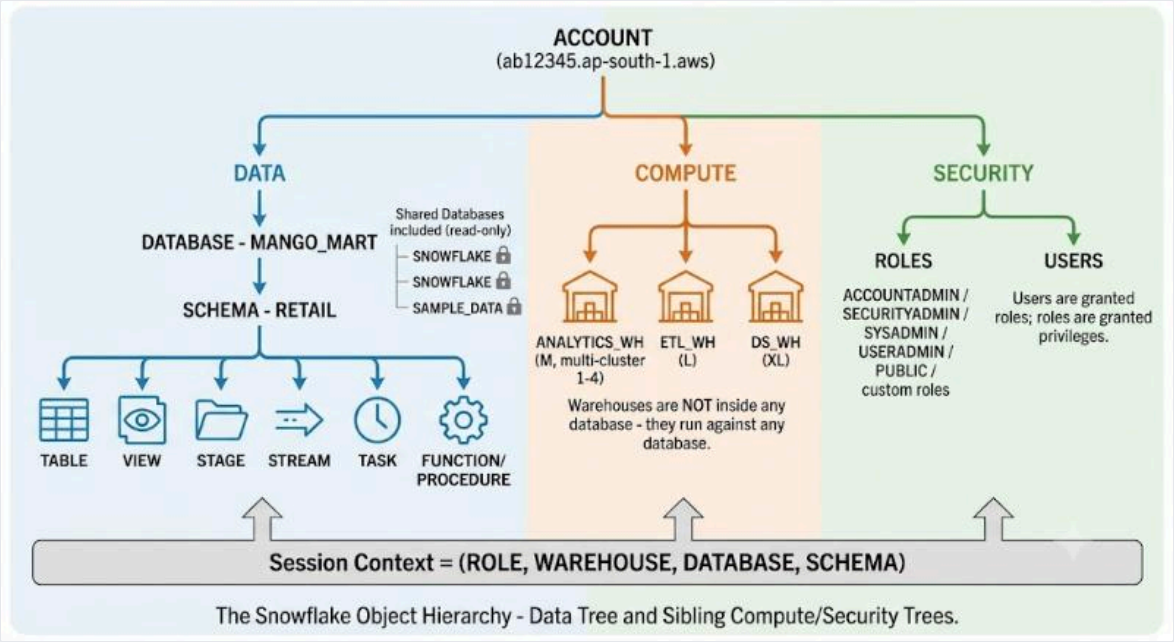
SUMMARY CARD

SnowSQL is Snowflake's Python-based command-line client - the primary way to script, automate, and CI/CD schema changes and data loads. Install once, configure named connections in `~/.snowsql/config` , and use `-f` to run files or `-q` for one-shot queries.

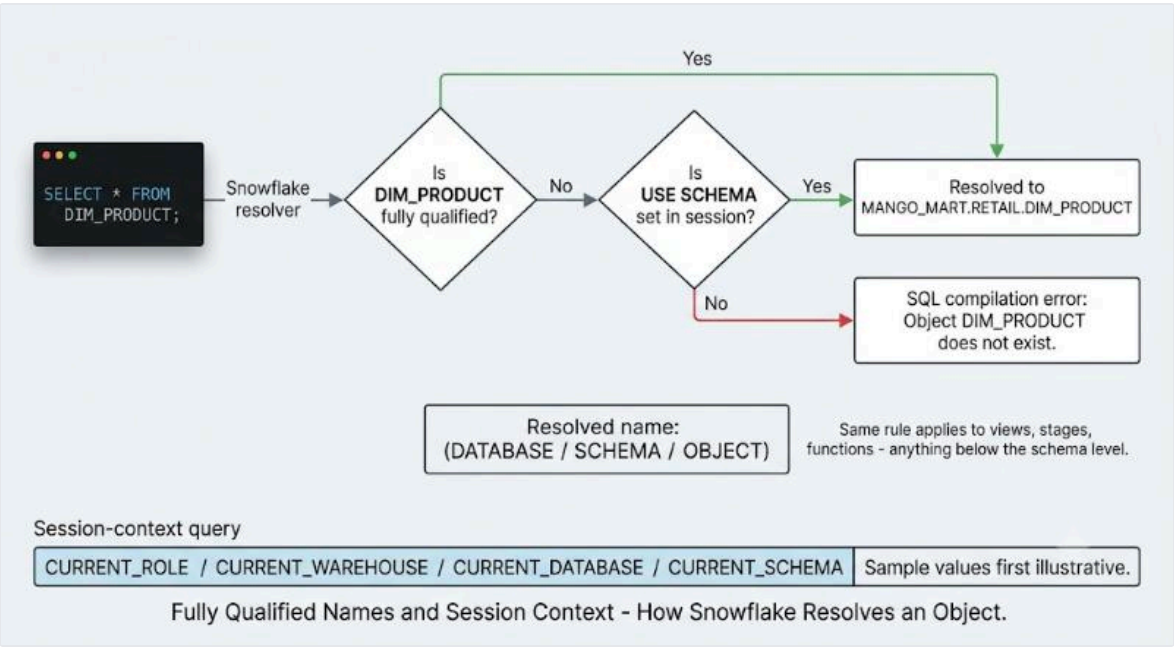
Together with Snowsight in the browser it forms the two-tool minimum every Snowflake practitioner keeps at hand.

THE OBJECT HIERARCHY - ACCOUNTS, DATABASES, SCHEMAS, TABLES, AND WAREHOUSES

Claude • AI Assistant



The Snowflake Object Hierarchy - Data Tree and Sibling Compute/Security Trees



Fully Qualified Names and Session Context - How Snowflake Resolves an Object

sql — Walk the entire object hierarchy of your account with SHOW commands

```sql
-- Top-down tour.
SHOW DATABASES;
SHOW SCHEMAS   IN DATABASE mango_mart;
```

```
SHOW TABLES    IN SCHEMA   mango_mart.retail;
SHOW VIEWS     IN SCHEMA   mango_mart.retail;
SHOW STAGES    IN SCHEMA   mango_mart.retail;
SHOW STREAMS   IN SCHEMA   mango_mart.retail;
SHOW TASKS     IN SCHEMA   mango_mart.retail;

-- Compute and security siblings.
SHOW WAREHOUSES;
SHOW ROLES;
SHOW USERS;

-- Zoom into a single object.
DESCRIBE TABLE     mango_mart.retail.dim_product;
DESCRIBE WAREHOUSE analytics_wh;
```

sql — Explicit session context at the top of every professional Snowflake script

```
-- Never rely on session defaults in production scripts.
-- Set every context variable explicitly.

USE ROLE      sysadmin;
USE WAREHOUSE etl_wh;
USE DATABASE  mango_mart;
USE SCHEMA    retail;

-- Sanity print - log this on every run.
SELECT current_role(), current_warehouse(),
       current_database(), current_schema(),
       current_region(), current_timestamp();

-- Now the real work.
CREATE OR REPLACE TABLE dim_product (
  product_id NUMBER(9) PRIMARY KEY,
  sku        VARCHAR(20),
  name       VARCHAR(200),
  category   VARCHAR(60),
  price      NUMBER(10,2),
  loaded_at  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Student

Sir, one thing has been nagging me. In MySQL I had 'databases' and 'tables' and that was it. In Snowflake I keep hearing 'account', 'database', 'schema', 'table', 'warehouse', 'role'. Can you give me the full picture of what contains what?

Claude • AI Assistant

Absolutely. This is the map that unlocks the whole platform.

Snowflake organizes everything into a **hierarchy** that is deliberately similar to a filesystem - parent folders contain child folders which contain files. But there is one important twist: **warehouses are not in the data hierarchy at all**.

They are compute. Let me draw the map, then we will walk through it level by level.

Student

Give me the map first.

Claude • AI Assistant

CONCEPT CARD

The **data hierarchy** - the tree of things that hold data - goes like this:

ACCOUNT (top) -> **DATABASE** -> **SCHEMA** -> **TABLE / VIEW / STAGE / STREAM / TASK / FUNCTION / PROCEDURE**

Alongside the data hierarchy, an **account** also holds **compute objects** (warehouses), **security objects** (users, roles, grants), and **operational objects** (resource monitors, network policies). Those live at the account level and are not nested inside databases.

Student

Walk me up the tree, please.

Claude • AI Assistant

Starting from the top.

- **Account.** Your Snowflake tenant. Identified by the account locator we saw in subsection 1.3 (`ab12345.ap-south-1.aws`). An account has exactly one region and one cloud

provider. Everything below belongs to this account. Billing happens at this level.

- **Database.** A logical container for related data. Contained by an account. Comparable to a database in Postgres. In our practice, `MANGO_MART` is our database.
- **Schema.** A container **inside** a database. Comparable to a schema in Postgres or Oracle - a namespace for tables. `MANGO_MART.RETAIL` is our schema. Every Snowflake object below the schema level uses the three-part name `DATABASE.SCHEMA.OBJECT` .
- **Table / View / Stage / Stream / Task / Function / Procedure.** The **leaf objects**. Tables hold data; views hold saved queries; stages hold files awaiting load; streams and tasks power change-data-capture pipelines; functions and procedures package reusable logic. All of these live inside a schema.

Student

Where do warehouses live?

Claude • AI Assistant

CONCEPT CARD

Warehouses live directly at the account level, not inside any database. This is a common point of confusion, and it flows directly from the storage/compute decoupling we discussed in subsection 1.2. A single warehouse - say `ANALYTICS_WH` - can run queries against **any database in the account**, provided the querying role has the right privileges.

That is why the four dropdowns at the top of a Snowsight worksheet are **independent**: role, database, schema, warehouse. You can freely mix and match.

Student

Give me a concrete address for a table.

Claude • AI Assistant

EXAMPLE CARD

Suppose I have loaded product data into a table called `DIM_PRODUCT` . Its **fully qualified name** is:

code — Code Example

```
MANGO_MART.RETAIL.DIM_PRODUCT
  |          |        |
```

```
|          |       +--- table
|          +----------- schema
+---------------------- database
```

If your **session context** has `USE DATABASE MANGO_MART; USE SCHEMA RETAIL;` set, you can just write `DIM_PRODUCT` and Snowflake resolves it. But in scripts, dbt models, and cross-schema queries, **always use the fully qualified name**.

It removes ambiguity and prevents bugs where a script accidentally reads from the wrong schema.

Student

What are those objects I have not seen yet - streams, tasks, stages?

Claude • AI Assistant

CONCEPT CARD

Quick preview - we cover each in depth in later chapters:

- **Stage** - a location that holds files (CSV, JSON, Parquet) waiting to be loaded into tables. Can be **internal** (Snowflake-managed) or **external** (your own S3/Azure/GCS bucket). Full detail in Chapter 2.
- **Stream** - a change-data-capture object that tracks inserts, updates, and deletes on a table since the last time you read it. Full detail in Chapter 4.
- **Task** - a scheduled or dependency-triggered SQL job that runs inside Snowflake, no external scheduler needed. Full detail in Chapter 4.
- **View** - a stored query. Two flavours: **standard view** (recomputed on each read) and **materialized view** (precomputed and refreshed automatically, Enterprise+).
- **Function** and **Procedure** - reusable logic. Functions return a value; procedures do work and can commit. Both can be written in SQL, JavaScript, Python, Java, or Scala. Full detail in Chapter 4.

Student

What about databases like `SNOWFLAKE` and `SNOWFLAKE_SAMPLE_DATA` ?

Claude • AI Assistant

Those are **shared databases**, shared by Snowflake into your account through the **Snowflake Data Sharing** feature. They occupy a slot in your database list but you cannot modify them.

You can, however, `SELECT` from them just like any other database - that is how the metadata views in `SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY` and the sample benchmark tables in `SNOWFLAKE_SAMPLE_DATA.TPCH_SF1` become accessible. Data sharing itself is a first-class feature of the platform, and it applies not just to Snowflake-owned data but to any producer sharing with any consumer.

Student

How do I actually see the tree in front of me?

Claude • AI Assistant

EXAMPLE CARD

Two ways.

In Snowsight: click the **Data** icon in the left sidebar -> expand databases -> expand schemas -> see tables, views, stages, streams, tasks. Click any object for its DDL, sample data, and lineage.

In SQL / SnowSQL - the `SHOW` and `DESCRIBE` commands walk the tree:

sql — Code Example

```sql
SHOW DATABASES;                    -- account -> databases
SHOW SCHEMAS   IN DATABASE mango_mart;
SHOW TABLES    IN SCHEMA   mango_mart.retail;
SHOW WAREHOUSES;                   -- account -> warehouses
SHOW ROLES;                        -- account -> roles

DESCRIBE TABLE     mango_mart.retail.dim_product;
DESCRIBE WAREHOUSE analytics_wh;
```

Student

How does **naming and case** work in Snowflake? Is `MANGO_MART` different from `mango_mart` ?

Claude • AI Assistant

> ⚠ **WARNING**
>
> This is one of the most common bugs new users create. **Snowflake stores unquoted identifiers in UPPER CASE and matches them case-insensitively.** So `mango_mart`, `Mango_Mart`, and `MANGO_MART` all refer to the same object.
>
> However, if you **quote** an identifier (`"mango_mart"`), Snowflake stores it exactly as written, case-sensitively. That means `"mango_mart"` and `"MANGO_MART"` are **two different objects**.
>
> The rule of thumb: **avoid quoted identifiers unless you have a compelling reason**. Stick to uppercase, unquoted names, and half the mysterious 'object does not exist' errors go away.

Student

What is a **role**, and how does it fit into this picture?

Claude • AI Assistant

> **CONCEPT CARD**
>
> A **role** is a bag of privileges - think of it as a job title. In Snowflake, permissions are never granted to users directly; they are always granted to **roles**, and users are granted **roles**. Snowflake ships with a few built-in roles:
>
> - **`ACCOUNTADMIN`** - top-level admin. Do not use for daily work.
> - **`SECURITYADMIN`** - creates users, roles, grants.
> - **`SYSADMIN`** - creates databases, schemas, warehouses. This is who owns your data objects.
> - **`USERADMIN`** - manages users and roles under `SECURITYADMIN`.
> - **`PUBLIC`** - every user gets this. Grants here are effectively public.
>
> For learning, do everything as **`SYSADMIN`** unless a specific step tells you otherwise. We devote a full subsection to RBAC in Chapter 3.

Student

One more - how do I switch context in the middle of a session?

Claude • AI Assistant

EXAMPLE CARD

Four `USE` verbs cover it:

sql — Code Example

```
USE ROLE      sysadmin;
USE WAREHOUSE analytics_wh;
USE DATABASE  mango_mart;
USE SCHEMA    retail;
```

You can also combine database and schema in one statement: `USE SCHEMA mango_mart.retail;` . And you can ask Snowflake what your current context is at any time:

sql — Code Example

```
SELECT current_role(), current_warehouse(),
       current_database(), current_schema(), current_region();
```

Make this five-column query a **muscle-memory habit** - run it whenever a query behaves unexpectedly, and 60% of the time you will find that you are in the wrong role, wrong warehouse, or wrong schema.

ANALOGY

Think of Snowflake as a large office building.

- The **account** is the building.
- **Databases** are floors.
- **Schemas** are rooms on a floor.
- **Tables, views, stages** are cabinets in a room.
- **Warehouses** are elevator systems shared across the whole building - they carry any authorized person to any room.
- **Roles** are ID badges - they say which floors and rooms you are allowed to enter.

Your **session context** (role + database + schema + warehouse) is you standing in a specific room with a specific badge, having taken a specific elevator.

REAL WORLD APPLICATION

I once helped debug a production incident where a nightly ETL job kept mysteriously loading into an empty schema. Root cause: the deploy script had one hidden `USE SCHEMA`

`"retail";` (quoted, lowercase) at the top, and the schema had originally been created as unquoted `RETAIL` (uppercase).

The two were different objects. Removing the quotes fixed it in one commit.

This exact bug will bite you at least once. When it does, remember: quotes preserve case.

COMMON PITFALL / MISTAKE

Beginners often create objects while accidentally in the wrong warehouse or wrong role. The object gets owned by the wrong role, and later steps in the pipeline (running as a different role) cannot see or modify it.

Prevention: at the top of every script, put explicit `USE ROLE`, `USE WAREHOUSE`, `USE DATABASE`, `USE SCHEMA` statements. **Never rely on session defaults in production.**

EXAM TIP CARD

SnowPro Core almost always includes questions like '*Which of the following is NOT contained by a schema?*' Answer: warehouses (they live at the account level). Also expect the reverse - '*A stage is contained by which object?*' Answer: schema.

PRACTICE QUIZ

What is the fully qualified name of a table called `SALES` in schema `PUBLIC` of database `ANALYTICS`?

A) `SALES.PUBLIC.ANALYTICS`

B) `PUBLIC.ANALYTICS.SALES`

C) `ANALYTICS.PUBLIC.SALES`

D) `ANALYTICS/PUBLIC/SALES`

▼ Reveal Correct Answer

Answer: C. Order is always DATABASE.SCHEMA.OBJECT.

EXERCISE

Log into your Snowflake trial. Run the five-column context query above.

Now change role to `ACCOUNTADMIN`, then back to `SYSADMIN`, and re-run it. Then switch warehouse to `COMPUTE_WH` and re-run again.

Notice how each `USE` statement changes exactly one field. Take a screenshot.

AI DEEP DIVE PROMPT

Ask an AI: '*Draw an ASCII diagram of the Snowflake object hierarchy, including where warehouses, roles, and stages sit.*' Compare its answer to the hierarchy above and note any errors.

SUMMARY CARD

Snowflake's world is organized as **ACCOUNT -> DATABASE -> SCHEMA -> object** for data, with **warehouses** and **roles** sitting directly at the account level as separate concerns. Every table has a three-part fully qualified name `DATABASE.SCHEMA.TABLE` .

Identifiers are stored uppercase and matched case-insensitively unless quoted. Your **session context** - role, warehouse, database, schema - determines what runs, where, and with which privileges.

Making these switches explicit at the top of every script is the mark of a professional Snowflake user.

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

sql — One-page snippet that summarizes Chapter 1 as executable SQL

```sql
-- Everything Chapter 1 taught, distilled into a single runnable block.

-- 1. Session context (subsection 1.5).
USE ROLE      sysadmin;
USE WAREHOUSE compute_wh;

-- 2. Data hierarchy (subsection 1.5).
CREATE DATABASE IF NOT EXISTS mango_mart;
CREATE SCHEMA   IF NOT EXISTS mango_mart.retail;
USE DATABASE mango_mart;
USE SCHEMA   retail;

-- 3. Decoupled compute - two independent warehouses (subsection 1.2).
CREATE OR REPLACE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE='MEDIUM' AUTO_SUSPEND=300 AUTO_RESUME=TRUE
  MIN_CLUSTER_COUNT=1 MAX_CLUSTER_COUNT=4;
```

```
CREATE OR REPLACE WAREHOUSE etl_wh
  WAREHOUSE_SIZE='LARGE'  AUTO_SUSPEND=60  AUTO_RESUME=TRUE;

-- 4. Fully qualified name (subsection 1.5).
CREATE OR REPLACE TABLE mango_mart.retail.dim_product (
  product_id NUMBER(9) PRIMARY KEY,
  name       VARCHAR(200),
  category   VARCHAR(60),
  price      NUMBER(10,2));

-- 5. Sanity check (subsection 1.5 - the muscle-memory query).
SELECT current_role(),     current_warehouse(),
       current_database(), current_schema(),
       current_region(),   current_timestamp();
```

CHAPTER SUMMARY

Yes - a clean recap is essential before we go deeper. In this chapter we built the **foundation** that everything else in this refcard rests on. Let me walk you through the five ideas, then hand you a checklist and a hands-on capstone that ties them together.

THE FIVE FOUNDATIONAL IDEAS

1. Cloud Data Warehouse (1.1). A fully managed, elastically scalable, columnar OLAP platform designed for analytical queries over very large datasets.

It replaces the fixed-capacity, high-maintenance world of on-prem warehousing with pay-as-you-go compute and effectively unlimited storage. Snowflake sits at the center of the modern data stack - every ingestion, transformation, BI, and ML tool eventually connects to it.

2. Separation of Storage and Compute (1.2). Snowflake's defining architectural bet.

One shared copy of your data in cloud object storage; many independent **virtual warehouses** on top. Two axes of scaling - **scale up** (bigger cluster, faster single query) and **scale out** (more clusters, more concurrent queries) - turn independently.

Multi-cluster shared-data is the phrase to remember.

3. Editions and First Setup (1.3). Four editions - Standard, Enterprise, Business Critical, VPS - layered as a feature superset.

Enterprise is the sweet spot for most projects and unlocks multi-cluster, materialized views, and long Time Travel. The 30-day, $400 free trial is a real production account you happen to be renting for free.

Snowsight is the modern web UI; `SNOWFLAKE_SAMPLE_DATA` and `SNOWFLAKE.ACCOUNT_USAGE` are the two default databases you will use every week.

4. SnowSQL CLI (1.4). The command-line client that turns Snowflake into a scriptable, CI/CD-friendly platform.

Named connections in `~/.snowsql/config`. `-f` runs files, `-q` runs one-shots, `-D` passes variables.

Interactive mode for exploration; non-interactive mode for cron, Airflow, and GitHub Actions. Snowsight and SnowSQL together form the two-tool minimum every practitioner uses.

5. The Object Hierarchy (1.5). `ACCOUNT -> DATABASE -> SCHEMA -> object` for data; **warehouses** and **roles** sit at the account level as separate concerns.

Every table has a three-part fully qualified name. Identifiers are uppercase and case-insensitive unless quoted.

The four `USE` verbs - `USE ROLE`, `USE WAREHOUSE`, `USE DATABASE`, `USE SCHEMA` - set your session context. **Make them explicit at the top of every script.**

ONE-SENTENCE CHAPTER DISTILLATION

Snowflake is a cloud-native OLAP platform whose defining move is to **decouple storage from compute**, exposed through a **familiar three-layer object hierarchy** you access via a **modern browser UI (Snowsight)** and a **scriptable CLI (SnowSQL)**, currently reachable through a **30-day free trial** that behaves exactly like a production account.

END-OF-CHAPTER CAPSTONE

> **EXERCISE**
>
> Complete all six steps end-to-end. Timebox: 90 minutes.
>
> 1. **Sign up** for Snowflake free trial - Enterprise edition, AWS Mumbai or Azure Central India.
> 2. **In Snowsight**, open a worksheet as `SYSADMIN` and run the starter setup that creates `MANGO_MART.RETAIL` (from subsection 1.3).
> 3. **In Snowsight**, create **two warehouses** - `ANALYTICS_WH` (Medium, multi-cluster 1-4) and `ETL_WH` (Large, single cluster) - using the code from subsection 1.2.
> 4. **Install SnowSQL** on your laptop. Add `[connections.mango]` to `~/.snowsql/config` (omit password). Connect: `snowsql -c mango`.
> 5. **From the SnowSQL prompt**, run the five-column `current_*()` context query from subsection 1.5. Then switch warehouse (`USE WAREHOUSE ETL_WH;`) and re-run. Screenshot both.
> 6. **Non-interactively** run a two-line file `hello.sql` containing `SELECT current_version(); SELECT current_account();` via `snowsql -c mango -f hello.sql`. Save the output.

If all six steps pass, you have successfully completed the ramp - you have an account, a starter database, two decoupled warehouses, a working CLI, and the muscle memory for switching context. Everything in Chapter 2 (architecture, warehouses in depth, data loading with `COPY INTO`) builds directly on this base.

VOCABULARY YOU SHOULD OWN BY NOW

OLTP, OLAP, MPP, columnar storage, cloud data warehouse, virtual warehouse, multi-cluster shared-data, scale up, scale out, edition (Standard/Enterprise/Business Critical/VPS), Snowsight, worksheet, SnowSQL, named connection, account locator, fully qualified name, session context, `USE ROLE / WAREHOUSE / DATABASE / SCHEMA`, `SNOWFLAKE_SAMPLE_DATA`, `ACCOUNT_USAGE`.

SNOWPRO CORE ALIGNMENT

This chapter contributes to two SnowPro Core exam domains:

- **Snowflake Data Cloud Features and Architecture** - separation of storage and compute, multi-cluster shared-data, editions.
- **Account and Security** - object hierarchy, session context, role basics.

Expect 4-6 questions on the exam that draw directly from Chapter 1 material.

WHAT'S NEXT

In **Chapter 2 - Snowflake Foundations**, we open the three-layer architecture in detail (cloud services, query processing, storage), unpack **micro-partitions** and pruning, tour **virtual-warehouse sizing** in production, and load your first real dataset into `MANGO_MART.RETAIL` using **internal stages** and the **`COPY INTO`** command. The vocabulary from this chapter - session context, roles, decoupled compute - will become the working currency of every code sample.

SUMMARY CARD

Chapter 1 gave you the **why** (cloud data warehouses replaced on-prem), the **how at the highest level** (decoupled storage and compute, multi-cluster shared-data), the **environment** (editions, Snowsight, SnowSQL, free trial), and the **map** (object hierarchy, session context). With those four in hand, the rest of the refcard becomes a set of concrete techniques rather than abstract new ideas.